10Pearls

Data Science Internship Programme

AQI Predictor

Forecasting the Air Quality Index for Lahore, 24 to 72 Hours Ahead

TECHNICAL REPORT

Submitted by

Imad Khan

Data Intern, 10Pearls
BS Data Science, Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

1 September 2026

Abstract

This report documents a serverless machine-learning system that forecasts the US EPA Air Quality Index for Lahore at 24, 48 and 72 hours ahead. Hourly weather and pollutant readings are collected by a scheduled job, the AQI is computed from raw concentrations using EPA breakpoints, features are stored in a managed feature store, models are retrained daily, and forecasts are served through both a JSON API and an interactive dashboard. No component runs on administered infrastructure.

The dataset spans 5.7 years and 49,153 hourly observations. Six forecasting approaches were evaluated — a persistence baseline, ARIMA, Ridge regression, Random Forest, gradient boosting and a neural network — across three horizons and three error metrics. **No standalone model beat the naive persistence baseline in any of the eighteen resulting comparisons.** The deployed forecast is consequently an anchored blend of a gradient-boosting model with persistence, which beats the baseline on R^2 and RMSE at all three horizons (0.832, 0.748 and 0.715 against 0.814, 0.709 and 0.628).

Two findings are of more general interest than the accuracy. First, the choice of evaluation protocol determined which model appeared best: under a single frozen train/test split Ridge regression outranked the tree models, and under walk-forward evaluation matching the daily-retraining deployment the ordering inverted completely. The same model scored 0.738 frozen and 0.831 walk-forward. Second, ARIMA — which uses no weather or pollutant features at all — outperformed every feature-based model under the frozen split, indicating that 33 engineered features contribute nothing beyond the series' own autocorrelation unless the model is retrained frequently.

The report also records five modelling hypotheses that failed, two quantitative errors found and corrected during preparation, and four production incidents in which a pipeline reported success while doing no useful work.

Contents

Figures

1. Problem, and what "solved" means here

Forecast the Air Quality Index for a city 24, 48 and 72 hours ahead, using only free tiers, with no server to administer. Lahore was chosen deliberately: its air quality is severe and highly variable, so the forecasting problem is real rather than a flat line to trace.

The requirement that shaped everything is easy to overlook. A 3-day forecast is not one number — it is three, at three different horizons, and each behaves differently. Near-term AQI is strongly autocorrelated and therefore easy; 72-hour AQI is not. Any honest evaluation has to report all three separately, and a model that wins at one horizon need not win at another. It turned out that assuming otherwise was the single costliest mistake made during this project (§6.3).

1.1 What was built

Component	Implementation
Feature pipeline	Hourly GitHub Actions job — fetch, compute EPA AQI, write one row to the feature store
Historical backfill	5.7 years loaded in monthly chunks, executed on a runner
Feature store	Hopsworks feature group, HUDI, composite key `(city_name, timestamp)`
Training pipeline	Daily GitHub Actions job — walk-forward evaluation, blend-weight fitting, registration
Model registry	Hopsworks, one model per horizon, blend weight stored beside the artifact
Serving core	One module assembling features and applying the blend, shared by both front ends
API	FastAPI — `/health`, `/current`, `/forecast`, `/models`, OpenAPI docs
Dashboard	Streamlit + Plotly, EPA colour scale, hazard alerts
Explainability	SHAP, regenerated weekly from the deployed model
Tests	72, no credentials required, run on every push

2. Data

2.1 Sources, and an asymmetry that shaped the design

Two providers, both free, and the split between them is not arbitrary. OpenWeather serves current weather, current air pollution, *and* historical air pollution at no cost — but its historical *weather* requires a paid plan. Historical weather therefore comes from Open-Meteo's archive API, which is free and needs no key. The backfill merges the two on the hour.

That asymmetry is worth stating because it is the reason the project has 5.7 years of data instead of 90 days. The free pollution archive reaches back to 27 November 2020, and finding that out changed the project's trajectory more than any modelling decision (§4.1).

2.2 Computing the AQI

OpenWeather returns raw pollutant *concentrations*, not an index, so the US EPA AQI is computed from scratch. Three details matter and each was a source of error:

1. **Unit conversion.** Gases are converted from µg/m^3 to ppb at 25 °C and 1 atm, with CO additionally converted to ppm:

   ```
   ppb = µg/m³ × 24.45 / molecular weight
   ```

2. **Truncation before matching.** EPA breakpoint tables have gaps between buckets — PM10 runs 0–54, then 55–154. An untruncated reading of 54.5 matches *nothing* and yields no sub-index. Concentrations are truncated to the EPA's specified precision first.
3. **Clamping at both ends.** OpenWeather's pollution figures come from a chemical transport model that occasionally emits small *negative* concentrations for trace gases. EPA breakpoints start at zero, so a negative reading matched no range and returned `None`, which then propagated into `max()`. Ninety days of data contained no such value; 5.7 years hit one immediately.

The overall AQI is the worst sub-index, and the pollutant producing it is recorded as `dominant_pollutant`. Tables use the 2024-revised PM2.5 breakpoints (Good ends at 9.0, not the obsolete 12.0).

> **KNOWN APPROXIMATION — DISCLOSED RATHER THAN HIDDEN**
>
> The EPA defines each breakpoint table over an averaging window: 24 hours for PM, 8 hours for O_3 and CO, 1 hour for SO_2 and NO_2. This implementation applies those tables to the single hourly reading OpenWeather publishes, because that is the only thing available at hourly cadence from a free source.
>
> The consequence is specific rather than vague. Because PM is officially averaged over a day and over one hour here, short pollution spikes register at full strength instead of being smoothed — so this value is *more volatile* than official EPA AQI, reading higher during a spike and lower after it. It tracks official AQI closely on flat days and diverges most on sharply changing ones. It is a consistent hourly proxy, correct as a forecasting target, and *not* a figure to publish as the official EPA AQI for a given hour. Fixing it means rolling concentrations to their proper windows before scoring; that is deliberately not done, because every model result in this report was measured against this definition and changing it silently would invalidate the comparisons.

2.3 What the store actually holds

Property	Value
Rows	**49,153** hourly observations
Span	2020-11-27 00:00 → 2026-08-15 02:00 UTC (5.7 years)
Expected hours in span	50,091
Missing hours	938 — **1.9%**
Duplicate timestamps	0
Nulls	0
Stored columns → model features	23 → 33
Mean / median / max AQI	243.3 / 189 / 500
Dominant pollutant	PM2.5 in **89.2%** of hours, ozone 10.4%, PM10 0.4%

The missing 1.9% is not cosmetic — it dictates how labels are built (§5.1). And the composite primary key `(city_name, timestamp)` makes inserts idempotent: re-running a backfill upserts rather than duplicating, which is what made recovery from a failed bulk load trivial rather than a data-cleaning exercise.

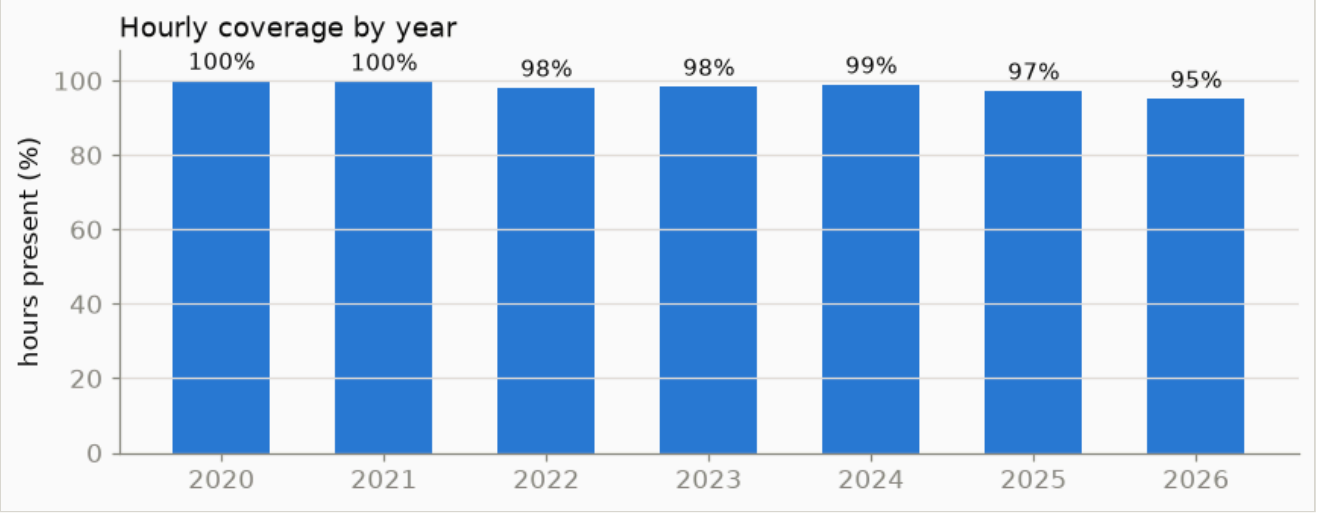

Figure 1 — Hourly coverage by year. Coverage is near-complete throughout. The gaps are dispersed rather than clustered, which is why interpolation over short windows is safe and why a positional label shift is not.

3. Exploratory analysis

Each finding below is followed by the design decision it forced. That ordering is the point: none of these were selected for interest after the fact.

3.1 The series

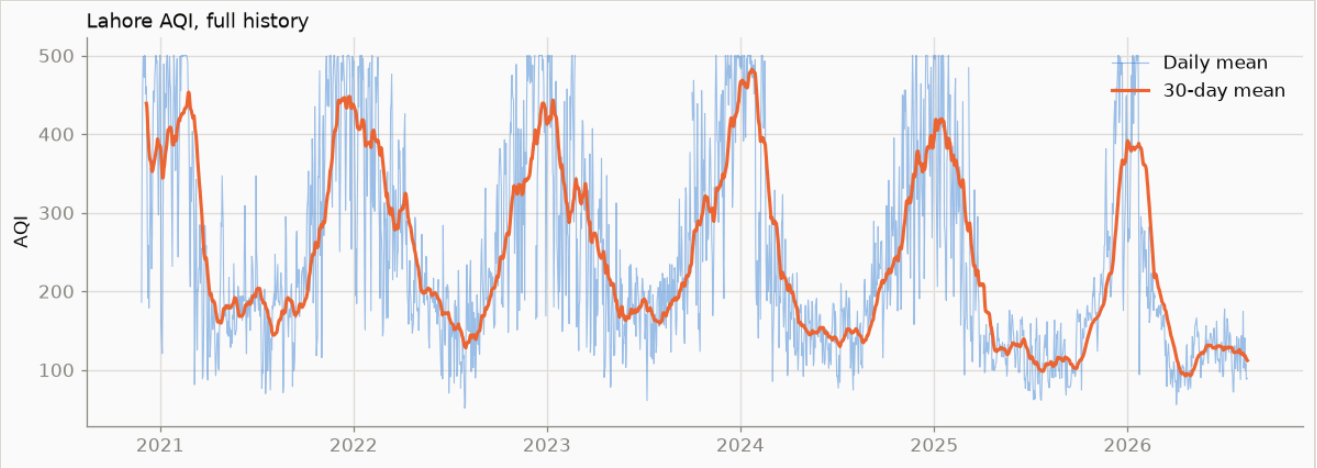

Figure 2 — Hourly AQI, 2020–2026, as a daily mean with a 30-day mean over it. Two structures dominate and they pull in opposite directions for a model: a strong annual cycle, and a downward drift across years. Sections 3.2 and 3.3 separate them.

Mean AQI is 243 and the median 189 — on the EPA scale that is "Very Unhealthy" on average. The distribution is wide (std 141, min 15, max the 500 ceiling) and right-skewed: the mean sits well above the median because winter spikes drag it up.

3.2 The trend, and a number that was wrong

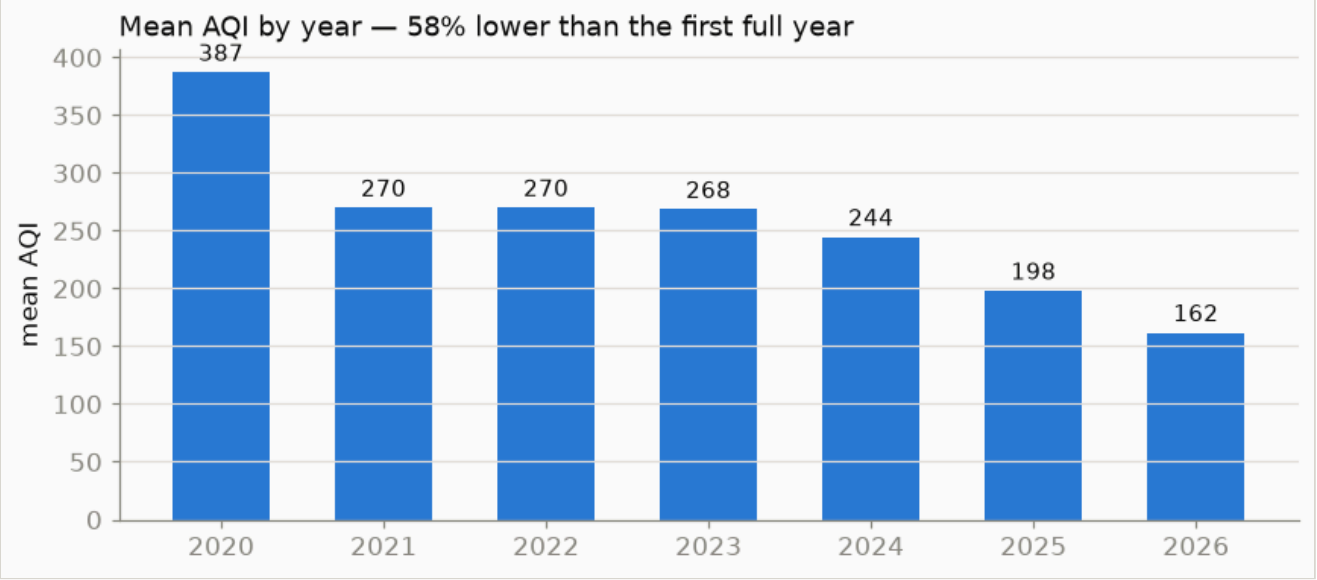

Figure 3 — Mean AQI by complete year. Grey bars are partial years and are excluded from the headline figure. Across complete years the decline is 270 → 198, about **27%**, or roughly 7% a year.

A CORRECTION, KEPT IN THE REPORT ON PURPOSE

For most of this project the decline was recorded as **58%** — 387 in 2020 against 162 in 2026 — and that figure propagated into the README, the experiment log, a chart title and two module docstrings. It is wrong. The archive opens on 27 November 2020, *inside* peak smog season, and ends in August, the cleanest month of the year. So the comparison put a winter-only stub against a summer-weighted one, with both biases pointing the same way.

Across complete years (2021–2025) the decline is **27%**. The modelling conclusions are unaffected — the regime shift is real, with a training-window mean of 262 against the test window's 169 — but the magnitude was overstated by more than double. The notebook now computes the figure across complete years only and greys out partial years with their coverage labelled on the bar, so it cannot be misread the same way again.

3.3 Seasonality, separated from the trend

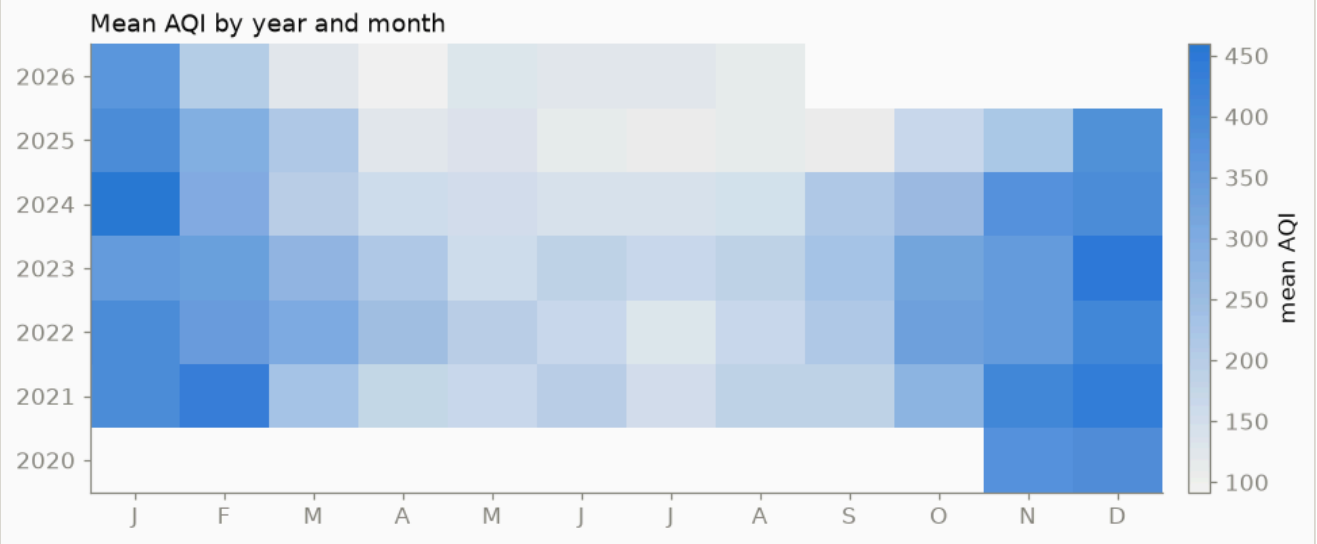

Figure 4 — Mean AQI by year and month. Reading across shows the seasonal cycle; reading down shows the trend. The improvement is concentrated in the clean season — summer months pale sharply across 2025–2026 — while **January 2024 is the darkest cell in the record**.

Monthly means range from 137 in July to 411 in December, with January at 392 and November at 343. This figure carries the most operationally important finding in the whole analysis: **winter smog has barely improved.** A headline "air quality is improving 7% a year" is true and, for a hazard-alerting system, close to irrelevant — the season when alerts matter is the season that has not improved.

Decision this forced. The test window must span whole seasons. An earlier evaluation used three weeks of stable summer and reported around MAE 19; a test window covering a winter reports around MAE 48 for the same class of model. The second number is not a regression, it is the honest one.

3.4 The daily cycle

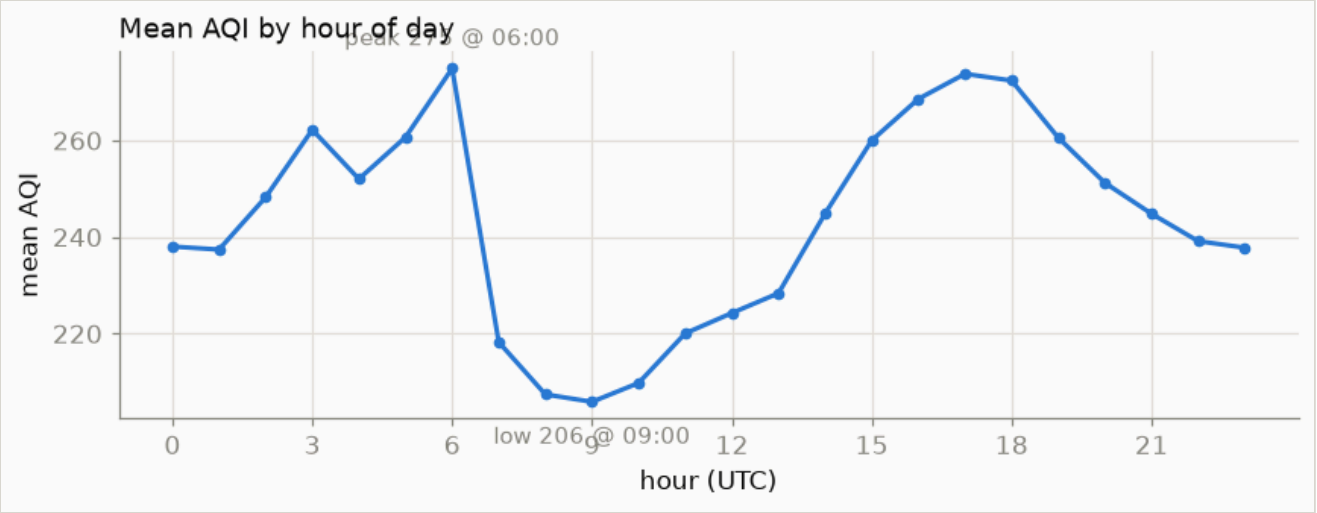

Figure 5 — Mean AQI by hour of day (UTC). A clear diurnal cycle driven by traffic and by the overnight collapse of the boundary layer.

Decision this forced. `hour` is kept as a feature and encoded cyclically as `sin/cos`, so 23:00 and 00:00 are adjacent rather than at opposite ends of a 0–23 range. `day`-of-month and `month` are computed and stored in the feature group — the brief asks for them — but **excluded from the model's feature set**, because including them measurably hurts (§5.3).

3.5 Why the naive baseline is so hard to beat

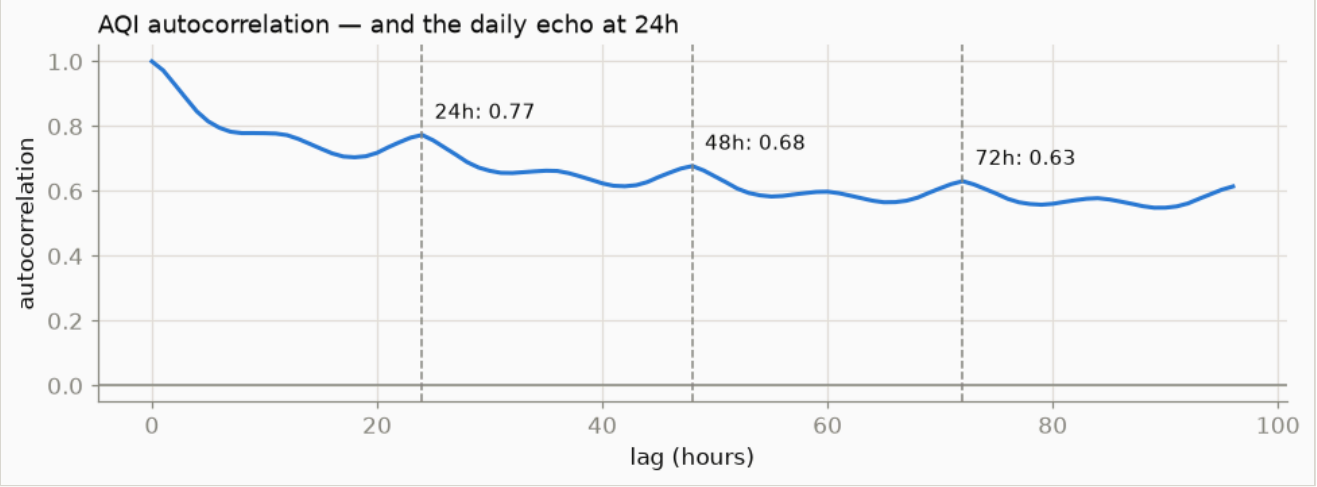

Figure 6 — Autocorrelation of hourly AQI with the three forecast horizons marked. **0.77 at 24h, 0.68 at 48h, 0.63 at 72h**, with a visible echo at multiples of 24 hours from the daily cycle.

This single figure explains the central difficulty of the project. AQI three days from now is still correlated 0.63 with AQI right now. Those three values line up almost exactly with the R^2 that "assume no change" achieves — 0.808, 0.704, 0.622 — which is not a coincidence but the same quantity viewed twice.

Decision this forced. **Persistence is scored alongside every model, every time.** A result not compared against it is not a result. Six models were compared against it and none won (§6.2).

3.6 Time spent in each EPA category

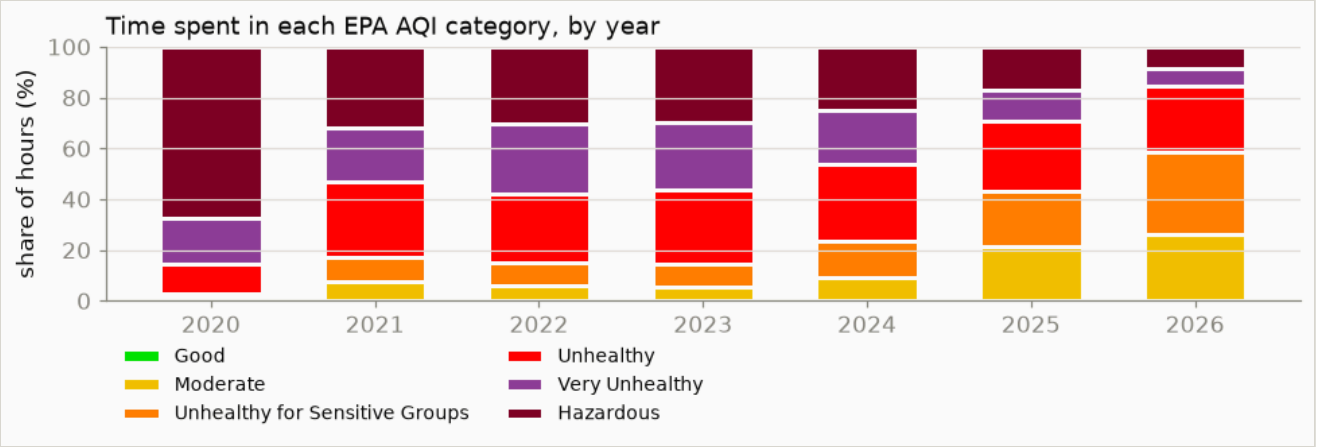

Figure 7 — Share of hours in each EPA category, by year, in the official AirNow colours used by the dashboard's alerts.

Hazardous hours fell from 67.3% of the 2020 window to 8.6% of 2026, while Moderate hours rose from 1.3% to 26.0%. This is the trend of §3.2 expressed in the units the system actually alerts on, and it is the most legible statement of the improvement available. Alert thresholds are read from the same `aqi_category()` table that produced these colours, so the alert boundaries and the AQI scale cannot disagree.

3.7 Which features carry signal

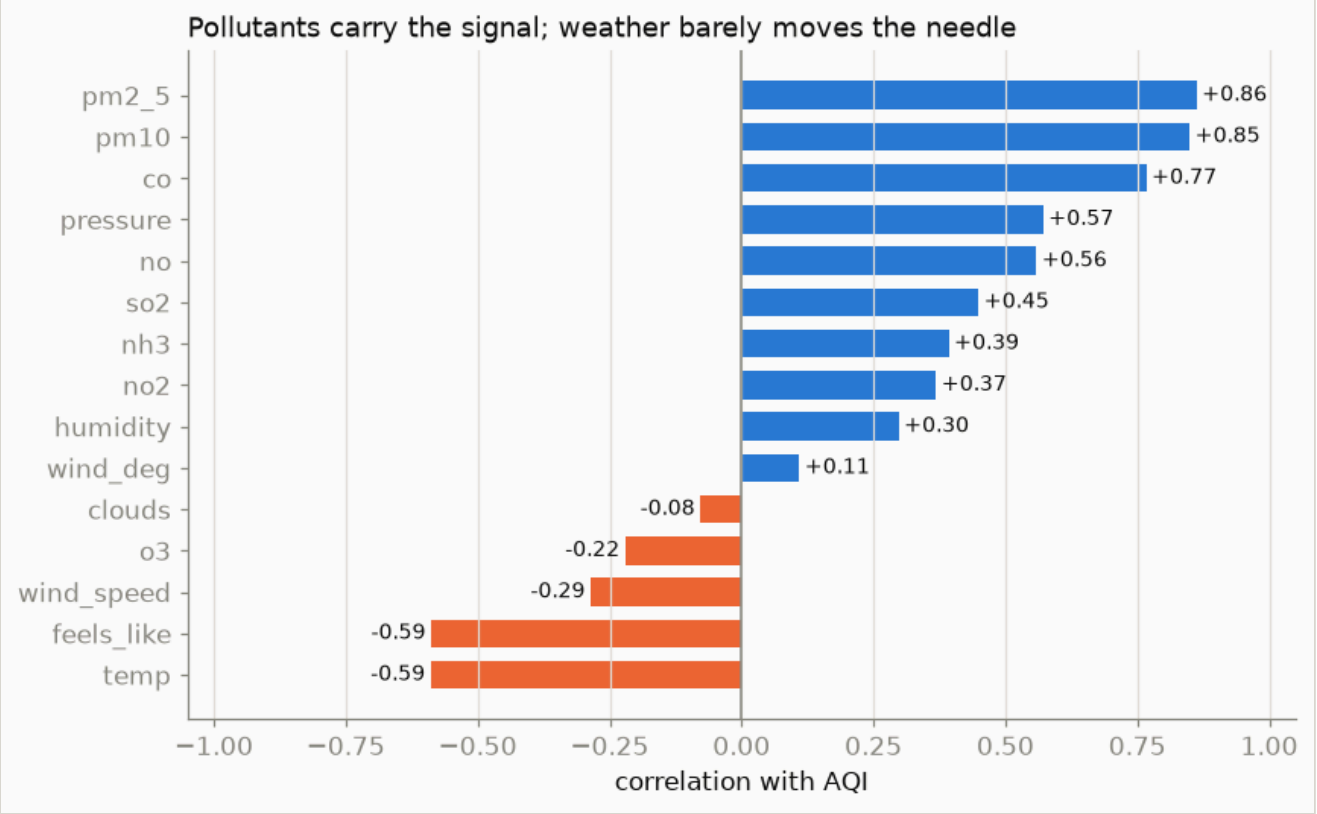

Figure 8 — Correlation of each reading with AQI. Pollutant concentrations dominate; weather barely moves the needle.

PM2.5 correlates strongly with AQI and sets the index in 89.2% of hours — unsurprising, since AQI *is* the worst sub-index and PM2.5 is usually it. Weather correlates weakly.

Decision this forced. Weather features were kept, but the weak correlation was an early warning that they carry less than hoped. It was confirmed later and expensively: supplying *forecast* weather as features made every horizon worse (24h R^2 0.718 → 0.695; 72h 0.519 → 0.368), so that idea was dropped on measurement rather than kept because it sounded sophisticated (§7).

3.8 Train and test are drawn from different distributions

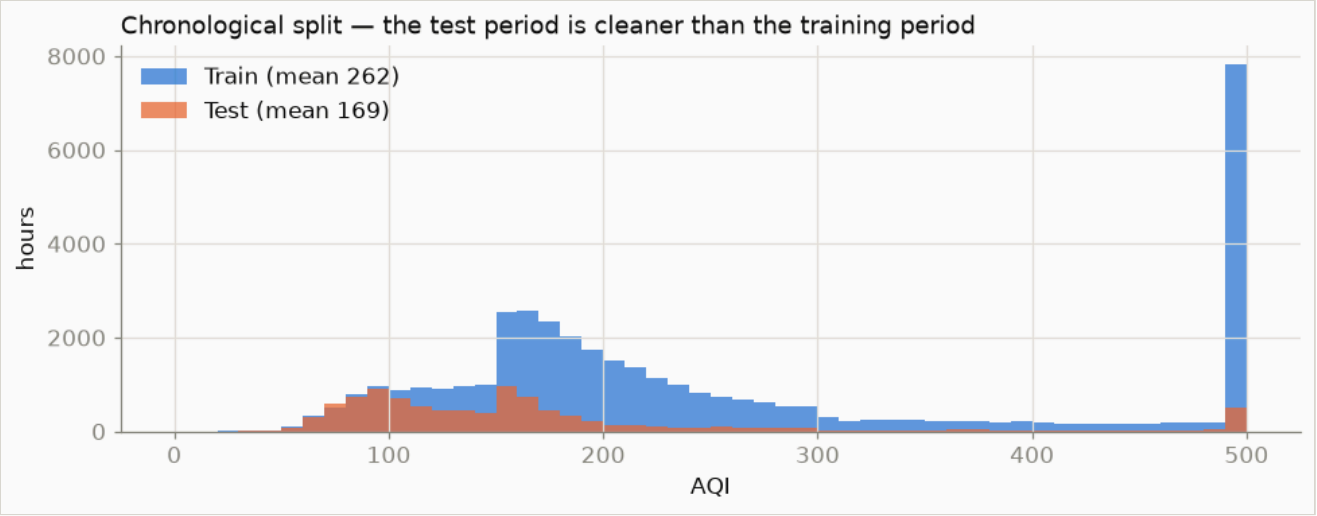

Figure 9 — AQI distribution either side of a chronological 80/20 split. Training window mean 262 (std 142); test window mean 169 (std 109). The test period is both cleaner and less variable.

A model fitted on the left distribution and scored on the right is being asked to predict a level it has never seen. This is the regime shift that dictates the target design (§5.2) and the evaluation design (§6.3).

4. From 90 days to 5.7 years

4.1 Diagnosing data starvation

The first backfill fetched 90 days — 2,065 rows — because a single request was issued for the whole range and nobody checked whether more history was available. A learning curve on that data was still climbing steeply at 100% of the sample, which is the signature of a model limited by sample count rather than by algorithm:

Training rows	R^2 at 24h
383	−0.328
766	−0.105
1,149	+0.010
1,532	+0.340

Probing the API directly showed the pollution archive reaches back to 2020-11-27 — roughly 24× more history, sitting unused. The backfill was rewritten to chunk by calendar month, insert in batches with retry, and cast explicitly to the feature group's locked schema.

4.2 What 24× more data changed

Three earlier conclusions reversed once there was enough data to test them properly, and this is the clearest evidence in the project that conclusions drawn on small samples are not conclusions:

Question	At 2,065 rows	At ~49,000 rows
Lag and rolling features	Harmful — R^2 0.645 → 0.211 at 24h	Helpful — 0.645 → 0.660 at 24h, 0.414 → 0.468 at 72h
`day` / `month` features	Load-bearing — removing them collapsed R^2 0.33 → −0.03	Harmful — 0.628 with, 0.645 without
72-hour horizon	Broken — R^2 −0.081	Working — +0.468
Neural network	Worst by far — R^2 −1.16	Mid-table — beats Random Forest and Ridge at 48h and 72h

The `day`/`month` row deserves emphasis. On 90 days of data, ablating those features destroyed performance, which reads as "they are important". They were important only because the model had learned almost nothing else: with 3.4 months spanning months 4 to 7, it was memorising "late July looks like this" rather than learning pollution dynamics. Same measurement, opposite interpretation, and only the larger dataset could tell them apart.

5. Features and target

5.1 Labels are matched by timestamp, never by row shift

To forecast *H* hours ahead, each row needs the AQI value from exactly *H* hours later:

```
future = df["timestamp"] + pd.Timedelta(hours=horizon)

df["aqi_target"] = aqi_by_time.reindex(future).values
```

Matched by timestamp arithmetic, not by shifting *H* row positions. With 1.9% of hours missing, a positional shift silently pairs a row with the wrong future value — a bug that raises no error and quietly corrupts every label downstream of a gap. This is the single most important line of data code in the project.

5.2 The model predicts a change, not a level

Given the regime shift of §3.8, a model predicting the AQI *level* learns the average of a dirtier era and systematically over-predicts today. Tree models make it worse: they cannot extrapolate below the levels present in training. So the models predict the **change**, and the forecast is anchored to the most recent reading:

```
prediction = current_aqi + w × predicted_delta
```

where `w` = 0 is exactly persistence and `w` = 1 the unshrunk model. Anchoring tracks the falling level for free, because `current_aqi` is today's measurement. The form is also, algebraically, a weighted average of the model with persistence — `(1−w)·aqi + w·(aqi + delta) = aqi + w·delta` — so deployment needs no separate persistence branch at inference time. `w` is fitted on validation and stored in a `blend.json` beside the artifact; it is never defaulted, because a default of 1.0 would silently ship the unshrunk model, which loses to persistence at every horizon.

5.3 The feature set

33 features: raw weather and pollutant readings; the computed AQI and its change rate; cyclically encoded hour; day-of-week; AQI lags at 1, 2, 3, 6, 12 and 24 hours; and rolling mean and standard deviation over 3, 24 and 72 hours. Every rolling window is shifted by one period first, so a row never sees its own AQI in its own statistics, and all of it is computed on a continuous hourly grid rather than by row position, for the reason in §5.1.

`day` and `month` are computed and stored — satisfying the requirement — but excluded from training on the evidence in §4.2.

5.4 What the model actually uses

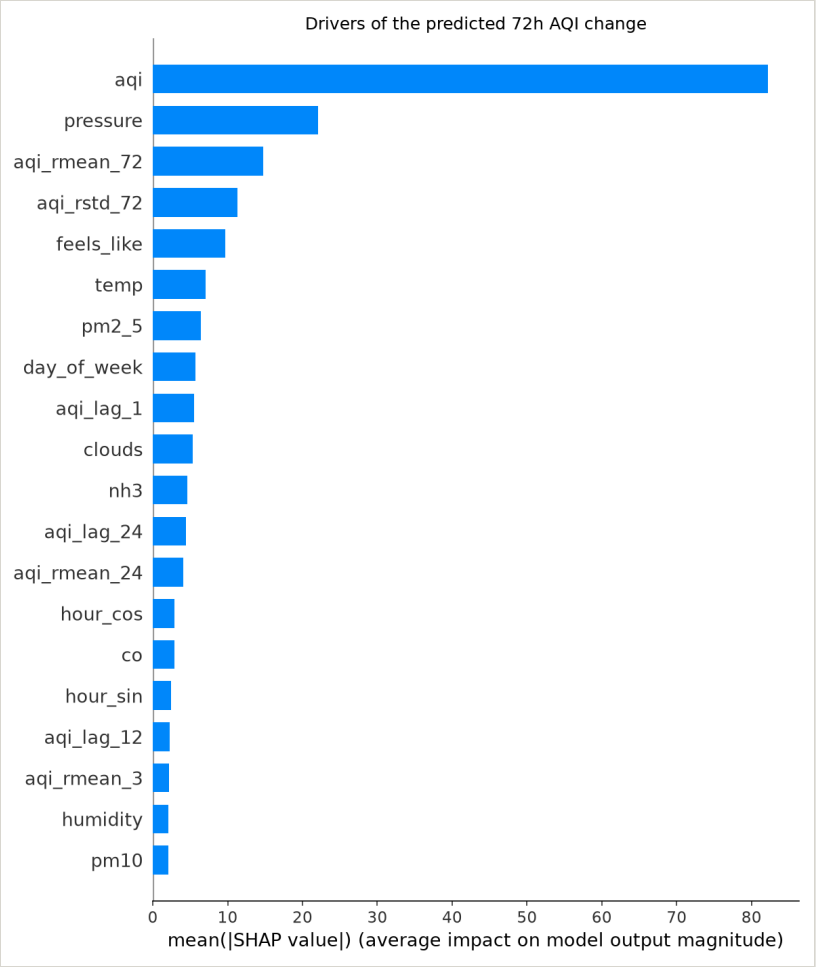

Figure 10 — SHAP importance for the deployed 72h model. Attributions are on the predicted *change* in AQI, not its level.

The current AQI dominates by roughly four to one over the next feature, and eight of the top twenty are AQI-derived (`aqi`, `aqi_rmean_72`, `aqi_rstd_72`, `aqi_lag_1`, `aqi_lag_24`, `aqi_rmean_24`, `aqi_lag_12`, `aqi_rmean_3`). Raw pollutant concentrations rank low despite PM2.5 correlating strongly with AQI — because AQI is already a feature, so PM2.5 is largely redundant given it.

`pressure` ranking second is the most interesting entry: it is a plausible proxy for atmospheric stagnation, which traps pollutants. It is also consistent with the ARIMA result in §6.2 — a model with access to nothing but AQI's own past does nearly as well as one with 33 features, because the AQI history is where most of the signal lives.

6. Method and results

6.1 Two leaks, and what they cost

Splits are chronological, never random: shuffling time-series data lets a model evaluate on rows whose near-identical neighbours it trained on. But chronological order alone is not enough. A training row at `T_split − 1h` carries a label from `T_split + 71h`, deep inside the test window, so the model sees test-period AQI during training.

Split at 72h horizon	R^2
Chronological only	0.326
Chronological + purge/embargo	**0.237**

About **27% of the apparent skill was leakage.** Every result in this report uses the purged split.

6.2 Six candidates against the baseline

All candidates predict the change in AQI and are anchored to the latest reading, so the numbers are directly comparable. Frozen purged split, all three metrics as the brief requires:

Model	RMSE 24h	MAE 24h	R^2 24h	R^2 48h	R^2 72h
Persistence (assume no change)	48.55	29.48	**0.808**	**0.704**	**0.622**
ARIMA(2,1,2) — AQI's own past only	52.21	33.08	0.778	0.669	0.590
Neural network — Keras MLP 32/16	57.49	44.91	0.730	0.627	0.576
HistGradientBoosting	57.39	41.99	0.731	0.488	0.517
Random Forest	60.60	45.24	0.700	0.489	0.504
Ridge regression	63.02	51.12	0.676	0.520	0.503

Eighteen comparisons, zero wins. Not one standalone model beats "assume no change" at any horizon. Two further observations from this table are more interesting than the ranking itself.

ARIMA — which sees no weather and no pollutant features at all — beats every feature-based model at every horizon. Under a frozen split, 33 engineered features add nothing over the series' own autocorrelation. They are not worthless: §6.3 shows they earn their place once the model is retrained frequently. But their contribution over plain autocorrelation is zero when the model is allowed to go stale, which is the strongest argument available that daily retraining is not a checkbox from the brief but the mechanism that makes the features pay.

The neural network went from worst to mid-table on an unchanged architecture. It scored −1.16 on 2,065 rows and was written off as hopeless. On 49,000 it beats both Random Forest and Ridge at the longer horizons. Nothing about the model changed except how much data it saw.

6.3 Changing the evaluation changed which model wins

Every result above trains once and predicts up to fourteen months into a frozen future. Production does nothing of the sort: it retrains *daily* and never forecasts more than 72 hours. Against a falling pollution level, the frozen split was mostly scoring models on their inability to predict a multi-year trend — a task they never face.

The replacement is walk-forward evaluation: retrain at successive monthly origins, score only the following month, roll forward. Twelve retrains, roughly 7,578 predictions per horizon, with persistence scored on identical rows.

> **THE FINDING THIS PROJECT IS ACTUALLY ABOUT**
>
> The same gradient-boosting model scores **0.738 under the frozen split and 0.831 walk-forward** at 24 hours. Nothing about the model changed — only whether the evaluation let it see recent data, as production does.
>
> More consequentially, **the ranking inverted.** Under the frozen split Ridge beat the tree models at every horizon; under walk-forward it is the *worst* and gradient boosting wins everywhere. Ridge only looked good because a stale model cannot track a falling trend and linear models extrapolate. Once retraining is frequent, that advantage evaporates and flexible models win on merit.
>
> An evaluation that does not match deployment selects the wrong model. This project selected the wrong model — Random Forest, chosen from a single bake-off run only at 72h and assumed to generalise to 24h and 48h — and shipped it, until the evaluation was fixed.

6.4 The deployed forecast

Persistence and the model make partly different errors, so a weighted average beats either. The weight is chosen on the previous month only, never on the month being scored:

Horizon	Persistence R^2	Model alone R^2	Blend R^2	Blend weight
24h	0.814	0.780	**0.832**	0.35
48h	0.709	0.643	**0.748**	0.35
72h	0.628	0.613	**0.715**	0.50

A fixed 0.5 scores within 0.005 of the tuned weight, which is reassuring: the gain is structural rather than the product of weight tuning.

6.5 Where the gain actually is — and where it is not

Reporting R^2 alone would overstate this result. All three metrics from the same run:

Horizon	RMSE blend	RMSE persistence	MAE blend	MAE persistence
24h	48.4	50.8	30.2	**29.8**
48h	59.0	63.5	38.9	39.0
72h	62.4	71.2	42.9	44.0

RMSE improves by 2.4, 4.5 and 8.8 points as the horizon lengthens. MAE is **marginally worse than persistence at 24 hours** (30.2 against 29.8), level at 48, and better only at 72. The blend suppresses *large* errors while leaving typical error roughly unchanged at short range.

For this application that is the right trade: the expensive mistake in a health-alerting system is missing a smog spike, not being three points off on a calm day. But it must be stated plainly, because the honest summary of the result is not "better at everything". Read the other way: **R^2 0.832 at 24 hours is not 0.832 of skill, because 0.808 of it is available from persistence alone.**

7. What did not work

Seven hypotheses were tested against persistence. Five failed or were marginal. They are recorded here because the two that worked are only credible given the ones that did not.

#	Hypothesis	Outcome
1	More data will close the gap	**Partly.** R^2 0.34 → 0.66 at 24h, but still below persistence. It exposed the real problem rather than solving it.
2	Predict the delta instead of the level	**No effect** under a frozen split — 0.703 against 0.697 at 24h. Only valuable in combination with the anchored blend.
3	Choose the model per horizon	**Real, but the winner depends on the evaluation.** Frozen split says Ridge; walk-forward says gradient boosting.
4	Train on a trailing window only	**Marginal.** Best 0.736 (24 months + Ridge) against 0.720 for all history. Short windows are disastrous — 3 months scores 0.036 — so the goal is to weight recent years, not discard old ones.
5	Add forecast weather as features	**Worse at every horizon.** 24h 0.718 → 0.695; 72h 0.519 → 0.368. Dropped.
6	Evaluate walk-forward instead of frozen	**Worked.** Closed most of the gap and, more importantly, changed which model wins.
7	Blend with persistence	**Worked.** Beat the baseline at all three horizons on R^2 and RMSE.

Hypothesis 5 is the one worth dwelling on. Feeding the model tomorrow's forecast weather sounds obviously helpful, and it made every horizon worse — badly so at 72h. Figure 8 had already hinted at why: weather carries little signal about AQI here, so the extra columns added variance without information. It was dropped on the measurement, not on the intuition.

8. Production engineering

8.1 The failure mode this project kept hitting

Eight separate times, something reported success while doing nothing useful, or reported a number that was wrong rather than missing. This is the most transferable lesson in the project, so each instance is listed with how it was caught. Note what the right-hand column never says: *an alert fired*.

Incident	What happened	Caught by
Automation contributed zero training rows	The live pipeline stamped rows with OpenWeather's observation time (e.g. 11:19:16) while the backfill wrote hour-aligned timestamps. Labels match at exactly *t*+24/48/72h, so an unaligned row can never match anything. Two weeks of hourly runs produced **0** usable examples; every daily retrain registered byte-identical metrics and reported success.	Querying the store directly: 2,100 rows, 2,065 hour-aligned, 35 not — and all 35 were the live ones.
Materialisation frozen for three days	Hopsworks writes land in Kafka and a background job files them into the queryable offline store. That job sat `INITIALIZING` for three days, so 49,080 inserted rows were invisible to reads. The pipeline said "inserted". CI was green.	Row count from `fg.read()` not matching what had been inserted.
Dashboard served a stale model	The training pipeline moved to a delta target with lag features; the dashboard kept its own copy of feature assembly, still loaded `aqi_random_forest_*`, and would have rendered a predicted *change* of −4 as "AQI 4 — Good" for a city sitting at 170.	Reading the serving path against the training code rather than trusting it.
Daily job dead for 24 hours	A commit changed `load_training_data` to return four values; `register.py` still unpacked three. The daily job failed instantly, at 02:00 UTC, in a tab nobody opens. The same change silently broke `explain.py`, which nothing imports.	Noticing the workflow history was entirely red while the hourly job was entirely green.
Materialisation frozen for eleven days	The same job as incident 2, wedged again from 2026-08-15 to 2026-08-26. The hourly pipeline inserted correctly and printed `Inserted feature row …` as its last line, three lines after a `UserWarning` saying the materialisation job was already running and the new execution had been aborted. **430 green runs.** The dashboard served an eleven-day-old reading under a caption reading "data refreshed hourly".	Reading the deployed dashboard as a stranger would, while reviewing three classmates' submissions for comparison.
A gap behind current rows	Clearing that stall did not fix the dashboard. Three fresh rows arrived on top of an eleven-day hole, so `MAX(timestamp)` was one hour old while serving — which needs 72 contiguous hours for its rolling features — correctly fell back to the last *complete* row, from before the gap. **The freshness assertion written that morning to catch exactly this class of failure passed**, because it measured the newest timestamp and never asked whether the hours behind it existed.	Two readings disagreeing: the pipeline reported the store current, the dashboard showed it eleven days stale. Both were right.
Eleven hours that had not happened	The backfill that closed the gap asked both source APIs for a range ending "today", and both return the remainder of the calendar day as *forecast*. Eleven rows dated 13:00–23:00 UTC were stored as observations. Left alone, the nightly retrain would have used forecast values as features *and* as labels.	The freshness check reporting `CURRENT` alongside a lag of **−11.0h** and coverage of **59/48 hours** — a verdict its own numbers contradicted.
The storage layer renamed the evidence	Hopsworks rewrites metric keys on write: `persistence_MAE` came back as `persistence_mae` and `persistence_R2` as `persistence__r2`, with a doubled underscore, while top-level `MAE` and `R2`	Inspecting the raw registry metrics after noticing the comparison columns were

	survived untouched. The dashboard's exact-match lookup found neither, rendered every baseline column empty, and concluded that a model beating its baseline "*beats baseline on: nothing*".	uniformly blank rather than mixed.

Green CI is not evidence. Every one of these was found by querying the thing that should have changed, not by reading a status badge. That is now written into the repository's conventions document.

8.2 Engineering decisions worth defending

- **One serving path.** After the stale-dashboard incident, feature assembly and the blend moved into a single module that both the API and the dashboard call. Two independent copies of that logic is exactly how the bug happened.
- **Models load lazily, not at startup.** A startup hook means a transient registry outage stops the service from booting; lazy loading degrades one retryable request instead.
- **`/health` never contacts the registry.** It reports cached state. A health check that can trigger a login becomes the load it is meant to detect.
- **Missing data returns 503, not 500.** The service is fine; the data is not there yet; the client should retry.
- **The blend weight ships in the API response.** A consumer cannot interpret the forecast without knowing how much of it is model and how much is persistence.
- **A horizon with no registered weight is withheld,** and named in `unavailable_horizons`, rather than served with a guessed one.
- **Every Hopsworks call is retried.** The registry write was the one that was not, and it is exactly where a run failed — HTTP 500, `errorCode 110043`, a ClusterJ `code 626 Tuple did not exist` raised inside Hopsworks' own filesystem metadata layer, after two of three artifacts had uploaded.
- **The loader walks model versions downwards** until one loads, because that failed write can leave a version whose metadata exists but whose files do not. Serving last week's intact model beats refusing to start.
- **Bulk operations run on a GitHub runner.** Sustained transfers from the development machine drop mid-stream — verified as a network limitation, not a code bug, by a 30-packet ping at 0% loss alongside a failed 10 MB upload.
- **Dependencies are pinned,** which immediately exposed that the runner had been silently installing TensorFlow 2.19.1 and NumPy 2.1.3 while development used 2.21.0 and 2.4.6. A dependency conflict that only appears once you pin is a conflict you already had.

8.3 Making the system say what it is doing

Seven of the eight incidents above were caught by a person reading something. That is not a monitoring strategy, so the last week of the project was spent making the system report on the state it is supposed to produce rather than on the call it just made.

- **The hourly job asserts its own outcome.** After inserting, it reads the offline store back and fails the run unless the newest row is recent *and* at least 80% of the last 48 hours are present. Two versions of this check were needed: the first looked only at the newest

timestamp and passed over an eleven-day hole; the second added coverage and still reported `CURRENT` on a negative lag. It now separates four states — current, stale, gapped, future-dated — because each has a different remedy and naming the wrong one costs an hour in the wrong web UI.

- **A read it cannot complete is a warning, not a failure.** Arrow Flight has its own outages, and losing an hour of collection to a monitoring call is the worse trade. Green must mean verified; amber means unverified; they are not the same claim.
- **Verdicts are published as run annotations,** not printed into a collapsed log step. `CURRENT: newest offline row 2026-08-27 02:00:00+00:00, 3.0h behind the insert, 45/48 hours present` appears on the run summary and in the checks API. A number nobody can find is not monitoring; it is a number.
- **Scheduled jobs check in to an external service.** Error reporting only fires when something throws, and for eleven days nothing threw. A check-in inverts that: the job reports that it started and finished, so *silence becomes the alarm*.
- **The dashboard states the age of what it is showing,** in words, with a warning past two hours and a red banner past a day. The caption that used to promise "refreshed hourly" now describes the pipeline's schedule, not the data's freshness — a claim a page makes about itself has to be falsifiable on that page.
- **Handled failures are still reported.** The dashboard deliberately serves the last good reading rather than a traceback when the feature store refuses a read. Swallowed is not the same as unnoticed, so that path sends the exception onward while the reader sees a clean page.
- **The pipeline attempts its own repair.** On a stale store it stops a materialisation execution that has been non-terminal for over 45 minutes and starts a fresh one — and *still fails the run*. Self-healing that hides the incident would recreate the original problem.

8.4 A scheduler is not a delivery guarantee

The 1.9% of missing hours in the dataset has a mundane cause that took until the final week to identify: **GitHub does not run scheduled workflows on time.** Measured firings of this repository's hourly schedule:

Date	Firings (UTC)	Count
2026-08-26	12:27, 13:43, 14:32, 16:06, 18:31, 21:21	6
2026-08-27	02:18, 13:11, 18:56	3

Gaps of up to five hours against a nominal one. Scheduled workflows are queued at low priority and dropped under load, and minute `0` — the obvious choice, and the one originally used — is the most contended minute on the platform. The cron was moved to minute 23, and the freshness tolerance was set to six hours for the same reason: an alert should mean the job has stopped, not that the platform is busy.

This also corrects an earlier reading of the same evidence. A stale store was diagnosed as a wedged materialisation job, which sent someone to the Hopsworks Jobs UI to find every execution green; the store was behind because the workflow had fired three times in seventeen hours. Two causes, one symptom. The failure message now names both and the annotation says which.

8.5 Testing

157 tests, none requiring credentials — the feature store, the model registry and the error reporter are all substituted, so a reviewer can clone the repository and run the suite with no accounts. Coverage is concentrated where failures have actually occurred: the EPA AQI calculation including negative concentrations and breakpoint gaps; the blend arithmetic and its clamping; withheld horizons; alert selection; schema drift; model-version selection and fallback; and the ARIMA path against a series with holes.

The more useful property is that the newer tests are named after real incidents and carry the real numbers. `test_the_eleven_day_stall_fails` feeds the check a 2026-08-15 offline timestamp against a 2026-08-26 insert. `test_the_real_24h_result_does_not_claim_an_mae_win` asserts that with R^2 0.832 against 0.814 but MAE 30.2 against 29.8, the dashboard reports a win on R^2 and RMSE and *not* on MAE. `test_the_mangled_baseline_keys_are_found` pins the exact keys the registry returned. A test that encodes a bug that actually happened is worth more than a test that encodes an intention.

9. Limitations

1. **The AQI is an hourly proxy, not official EPA AQI.** Breakpoints are applied to instantaneous readings rather than their proper averaging windows (§2.2). More volatile than the official index; correct and consistent as a forecasting target.
2. **MAE at 24 hours is marginally worse than persistence** (§6.5). The gain is in large errors, and it grows with horizon.
3. **Absolute error is wide.** MAE of 30–43 AQI points can straddle an EPA category boundary, so alerts are directionally useful but not precise near thresholds.
4. **ARIMA(2,1,2) does not always converge.** statsmodels emits `ConvergenceWarning` at longer horizons. The ranking is stable and the forecasts usable, but it is a real limitation of the statistical baseline.
5. **One city, one pollutant regime.** Nothing here has been tested outside Lahore, where PM2.5 sets the index 89% of the time. A city dominated by ozone would likely need a different feature set.
6. **The forecast horizon is capped by the anchor.** Anchoring to the current reading is what makes the trend tracking work, and it is also why this design would not extend to week-ahead forecasting.
7. **Free-tier dependence.** Two outages of the Hopsworks Query Service blocked analysis during this project. The system degrades correctly, but a submitted analysis should not depend on a service being up; a committed data snapshot is the outstanding fix.
8. **The hourly job over-installs.** It installs the full data-science stack 24 times a day and imports six packages. Known, not yet fixed.

10. What I would do next

1. **Commit a feature snapshot** so the EDA and the model comparison are reproducible without credentials or a live service — for a reviewer as much as for resilience.

2. **Weight recent years rather than truncating.** Hypothesis 4 showed short windows are harmful and long ones dilute; sample weighting decaying with age is the obvious middle path and was not tested.
3. **Roll concentrations to their EPA averaging windows** and re-measure everything against the corrected target, reporting both.
4. **Predict the winter peak specifically.** §3.3 shows winter has not improved and is when alerts matter. A model trained or evaluated on smog season alone is a more useful product than one averaged across the year.
5. **Quantile forecasts rather than point forecasts.** A health alert wants "80% chance of exceeding 300", not a single number with an implicit ±40.

11. Requirement coverage

#	Requirement	Where
1	Fetch raw weather and pollutant data from an external API	`feature_pipeline/fetch.py` — OpenWeather
2	Compute features and targets; time features (hour, day, month) and derived features such as AQI change rate	`feature_pipeline/pipeline.py`, `training_pipeline/data.py` — all stored; `day/month` excluded from training on measurement (§4.2)
3	Store features in a feature store	Hopsworks — `feature_pipeline/hopsworks_store.py`
4	Backfill historical features and targets	`backfill/` — 49,153 rows from 2020-11-27
5	Training pipeline: read, train, evaluate, register	`training_pipeline/`
6	scikit-learn (Random Forest, Ridge) and TensorFlow/PyTorch	Ridge, Random Forest, HistGradientBoosting, Keras MLP, ARIMA — §6.2
7	Evaluate with RMSE, MAE and R^2	`evaluate.py` — all three, every model, every horizon
8	Store the model in a model registry	Hopsworks registry — `register.py`
9	CI/CD: features hourly, training daily	`.github/workflows/` — plus tests on push and explainability weekly. Scheduled hourly; GitHub delivers 3–7 firings a day (§8.4)
10	Web app loading model and features, descriptive dashboard	`app/app.py` — Streamlit + Plotly
11	Streamlit/Gradio *and* Flask/FastAPI	`app/app.py` and `api/main.py`, over one serving core
12	EDA to identify trends	`notebooks/01_eda.ipynb` — §3
13	A variety of models, statistical to deep learning	ARIMA through Keras MLP — §6.2
14	SHAP or LIME feature importance	`training_pipeline/explain.py` — §5.4; ranking published as data and rendered interactively with readable feature names
15	Alerts for hazardous AQI levels	`serving/forecast.py`, thresholds from the same table as the AQI scale
16	A detailed report	This document

12. Reproducing this

```
git clone https://github.com/imadkhan4478/AQI_Predictor.git

python -m venv .venv && .venv\Scripts\activate

pip install -r requirements.txt

pip install -r requirements-deep.txt  # optional: the neural-net row



pytest                              # 72 tests, no credentials needed

uvicorn api.main:app --reload    # API + docs at /docs

streamlit run app/app.py         # dashboard
```

The pipelines and the analysis need six environment variables, documented in `.env.example`; they are held as GitHub Actions secrets and no secret has ever entered the repository, verified by a full history scan. Anything reading the whole feature group should be run through the **Analysis** workflow rather than locally, for the transfer reason in §8.2.

Every figure in this report was produced by `notebooks/01_eda.ipynb` or `training_pipeline/explain.py` against the live feature store, and every number was read from an executed run rather than carried forward from an earlier draft. Two figures were found to be wrong during that check and are corrected here: the pollution decline (§3.2) and the proportion of missing hours. The chronological record, including the failed attempts in the order they happened, is `docs/EXPERIMENT_LOG.md`.